



Глава 1. Основы

Ежелев Г. И.

11 июля 2026 г.

О чем эта глава?

Предполагается, что вы уже чуть-чуть знакомы с C++ или подобными языками. Объяснять вам что такое `for`, `while` и т.д. в десятый раз не будем.

В этом разделе будут рассказаны не самые очевидные вещи, которые нам понадобятся дальше на всех занятиях от сентября до мая:

- ▶ Особенности некоторых типов данных и их внутреннее устройство
- ▶ Подробнее про массивы и указатели
- ▶ Процесс компиляции программ и сборки проектов
- ▶ ООП
- ▶ Стек, очередь и связные списки
- ▶ Стек вызовов

Таблица 3.1. Основные типы данных в C++. Типичные размеры значений и диапазоны представления

Тип	Размер в битах	Диапазон
char	8	-127-127 или 0-255
wchar_t	16	0-65 535
int (16-разрядная среда)	16	-32 768-32 767
int (32-разрядная среда)	32	-2 147 483 648-2 147 483 647
float	32	3,4E-38-3,4E+38
double	64	1,7E-308-1,7E+308
bool	-	true или false
void	-	Без значения

- ▶ Вы вероятно уже не раз видели эту таблицу.
- ▶ Для целочисленных типов(int, uint, byte, long) есть более удобный способ записи:
 - int8_t, int16_t, int32_t, int64_t для знаковых типов
 - uint8_t, uint16_t, uint32_t, uint64_t для беззнаковых типов

Хранение целых чисел

Внутри любого CPU есть устройство ALU (Arithmetic and Logic Unit). В нем реализована схема сложения двоичных беззнаковых чисел.

Вопрос: как хранить в памяти знаковые числа так, чтобы все арифметические операции можно было делать с минимальным изменением ALU?

Рассмотрим на примере 16-битных чисел:

- ▶ Знаковый бит. Пусть старший бит будет отвечать за знак (1 - минус, 0 - плюс). Тогда возникают проблемы:
 - 1) Два нуля: 1000...0 и 0000...0
 - 2) Нужна отдельная схема в ALU для вычитания
- ▶ Обратный код. Отрицательное число - это инверсия всех битов положительного. Пример число 6 (0110) $\Rightarrow$ -6 - это 1001. Тогда при сложении A и -A будет получаться 0 и нам не понадобится лишние схемы в ALU!
Есть проблема: всё ещё 2 нуля: 000...0 и 111...1

- Дополнительный код. Отрицательное число - это дополнение исходного числа до максимального + 1.

На первый взгляд звучит сложно (и кажется, что понадобятся сложные схемы для реализации).

Что такое макс. число в случае 16-битных ячеек? Это $2^{16} - 1$, т.е. 111...1.

Тогда дополнение $\bar{A}$ числа A - это инверсия битов. Проверьте это!

Тогда доп. код числа A - это $\bar{A} + 1$ - обе операции (инверсия и +1) уже есть в стандартном ALU => сложность конструкции не увеличится.

•Проверка:

$$\forall A + (\bar{A} + 1) = (A + \bar{A}) + 1 = (2^{16} - 1) + 1 = 0$$

Последнее равенство корректно, ведь 2^{16} это $1\underbrace{000...0}_{16 \text{ нулей}}$ и 1 в ячейку не поместится.

16 нулей

2СС и 16СС

2СС и 16СС очень просто переводятся друг в друга:

Цифра 16сс	Число 2сс	Цифра 16сс	Число 2сс
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Если хотите попрактиковаться дома(очень советую)

- $2_{CC} \rightarrow 16_{CC}$ и 10_{CC}

111_2 ; 1100101_2 ; 101101_2 ;

110101_2 ; 11011010_2

- $16_{CC} \rightarrow 2_{CC}$ и 10_{CC} (из 16_{CC} напрямую в 10_{CC} , не используя 2_{CC})

$7A_{16}$ $1C9_{16}$ $5D0F_{16}$ $A2B3_{16}$ $B4E_{16}$

- Вычислите в 2_{CC} числа: -73_{10} -18_{10} -45_{10} $-3A_{16}$
 $-1F_{16}$

Char почти всегда соответствует набору символов ASCII - American Standard Code for Information Interchange (код обмена информацией). Это один из первых текстовых кодов. Занимает ровно 8 бит. Первые 30 символов служебные, т.к. ASCII кодирует текст по принципу печатной машинки (например есть символы сдвига курсора на один символ назад (можно писать символы поверх друг-друга) и переноса строки и каретки для новой строки). При работе с экраном нам понадобится это великое знание.

Цикл range for

for (объявление : диапазон) оператор;

```
for (int n : {0, 1, 2, 3, 4, 5}) {  
    std::cout << n << ' '  
}  
std::cout << '\n';
```

Цикл do-while

***do
оператор;
while (выражение);***

```
unsigned long n;  
do  
{  
    std::cout << "Enter number (0 to end): ";  
    std::cin >> n;  
    std::cout << "You entered: " << n << std::endl;  
} while (n != 0);
```

- ▶ Указатель(pointer) - это переменная, область значений которой - адреса ячеек памяти + специальное значение - нулевой адрес
- ▶ Указатель указывает на некоторую ячейку памяти. Через него можно обратиться к этой ячейке.
- ▶ Нулевой адрес - показывает, что сейчас указатель никуда не указывает
- ▶ &X - Получить адрес объекта X
- ▶ *Y - Обратиться к ячейке по адресу Y

Указатели. Операторы & и *

```
int x = 1;
int y = 2;
int z[10];
int* ip;      /* ip - указатель на int */

ip = &x;      /* теперь ip указывает на x */
y = *ip;      /* y теперь равен 1 */
*ip = 0;      /* x теперь равен 0 */
ip = &z[0];   /* ip теперь указывает на z[0] */
```

Связь массивов и указателей. Арифметика указателей

STM32

Глава 1.

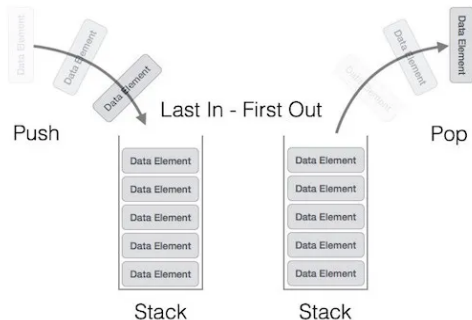
Основы

Ежелев Г. И.

- ▶ Элементы массива располагаются в памяти последовательно - один за другим.
- ▶ Над указателями можно производить обычные арифметические операции
- ▶ Имя массива - указатель на первый элемент.
- ▶ Запись `mass[i]` преобразуется компилятором в `*(&mass[0] + i*sizeof(obj))`.
- ▶ Массив можно объявлять как указатель и пользоваться им как массивом.

Стек (Stack)

- ▶ Структура данных - это некоторый способ организовать хранящиеся данные умнее и эффективнее, чем просто массив.
- ▶ Нам далее понадобятся стек, очередь, связный список и граф.
- ▶ Стек реализует подход LIFO (Last In, First Out)
- ▶ **Задание:** Реализуйте стек на массиве





Мотивация

- Зачем?



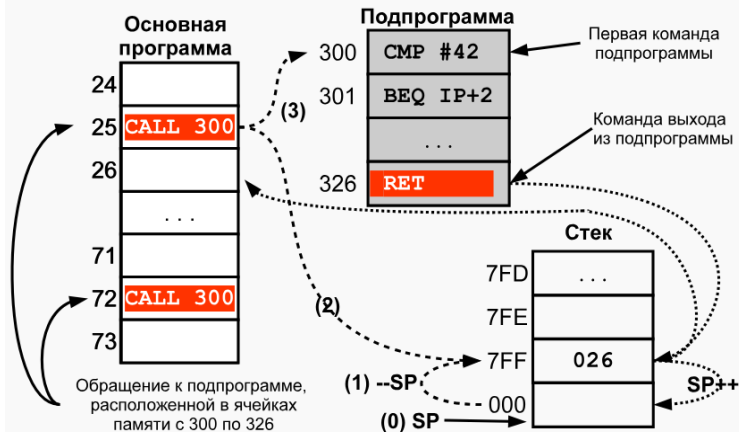
- Когда выгодно создавать подпрограмму?
 - Размер кода
 - Передача и обработка параметров; возвращение результатов
- Инлайнинг — процесс, обратный выделению кода в подпрограмму

28

- ▶ Для инлайнинга в C++ есть ключевое слово `inline void f(){...}`
- ▶ Инлайнинг экономит время перехода к подпрограмме, сохраняя лаконичность кода.



БЭВМ: Вызов программы и возврат из нее



(пояснение слайда далее)

- ▶ Для вызова подпрограммы (в примере по адресу 300) нужно записать адрес откуда мы прыгнули в подпрограмму (чтобы потом вернуться обратно).
- ▶ Для этого прекрасно подойдет стек.
При вызове подпрограммы на вершину стека будет помещен адрес возврата.
В конце подпрограммы на вершине стека будет лежать самый последний адрес возврата - он нам и нужен.
- ▶ Такая архитектура прекрасно подойдет для вложенных вызовов и рекурсии
- ▶ Указатель на стек называют SP - Stack Pointer.
Если в стеке закончится место - возникнет ошибка Stack Overflow (переполнение стека).

Стек вызовов. Передача аргументов

STM32

Глава 1.

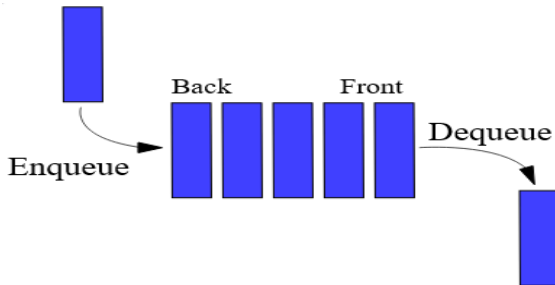
Основы

Ежелев Г. И.

- ▶ Аргументы подпрограмме можно передавать через заранее определенные ячейки памяти
- ▶ А можно загружать их в стек до вызова подпрограммы. Тогда все аргументы будут лежать в стеке под адресом возврата
- ▶ Никто нам не мешает внутри функции "подсмотреть" что лежит под вершиной стека. Удалять конечно, как при обычном чтении что-то под вершиной стека нельзя.
- ▶ Описанная структура - стек вызовов. Используется в большинстве языков программирования.
- ▶ Передача аргументов через стек - стандарт.

Очередь (Queue [кью])

- ▶ Структура данных, организованная по принципу FIFO - First In, First Out
- ▶ Нужна для организации последовательности выполнения действий.
- ▶ Она нам понадобится в алгоритмах на графах, чтобы обозначать порядок обхода вершин, а также в роботах-футболистах для формирования сложных траекторий движения на поле.
- ▶ **Задание:** Реализуйте очередь!



- ▶ Сборка проекта - процесс превращения программы в машинный код. Разберем его по этапам:
- ▶ Препроцессинг (Preprocessing): Препроцессор работает с программой исключительно как с текстом.
 - Препроцессор обрабатывает директивы препроцессора - команды, начинающиеся с #:
 - 1) `#define a b` - заменить все строчки "a" на все строчки "b" это НЕ ПЕРЕМЕННАЯ!!!
 - 2) `#include abc.(h/cpp/...)` - вставить целиком весь текст указанного файла
 - 3) `(#ifdef/#ifndef abc) ... #endif` - вырезать программу, если есть/нет `#define abc`
 - 4) `#pragma ...` - указание какого-то специфического действия с файлом

- ▶ Компилятор - преобразовывает каждый файл по-отдельности, пропущенный через препроцессор в набор бинарных объектов. На выходе каждый .cpp файл превращается в .obj файл.

Каждый объектный файл содержит: Секцию кода (команды), Секцию данных и таблицу символов (Symbol table) - все вызываемые объекты (переменные, классы и т.п.).

- Некоторые объекты объявлены в других файлах (например методы внутри библиотек) - тогда нам нужна ссылка на этот символ.
- Если символ объявлен внутри файла и он может использоваться другими файлами, тогда он объявляется в .h файле - это заголовочный файл (header file), а в исходный .cpp файл добавляется строка `#include [название заголовочника].h`

- Создание .h файлов в разы ускоряет компиляцию (нам нужно связывать ссылки только на сами объекты, а не всю реализацию). Например, если была изменена функция в файле `gpio.h`, то нужно перекомпилировать только его, а все файлы, использующие `gpio.h` из-за ссылок, а не прямого копирования кода из `gpio.h` перекомпилировать не надо.

- ▶ Ассемблирование - перевод машинных команд в коды, понятные процессору (можно считать частью этапа компиляции)
- ▶ Линковка (Linking) - как раз связывание всех файлов (их ссылок) воедино + расположение программы в памяти МК.

частая ошибка: Если видите ошибку линковки - вероятно линковщик не может найти в вашем файле какой-то объект, или наоборот, нашел несколько объектов с одним именем в разных файлах.